

1. (BM vs. RBM) Demostración de la Complejidad de Inferencia

Demuestre matemáticamente por qué la inferencia para una unidad oculta (calcular $P(h_j = 1|v)$) en una Máquina de Boltzmann (BM) general es computacionalmente intratable, mientras que en una Máquina de Boltzmann Restringida (RBM) es tratable y eficiente.

Claro, te ayudaré a demostrar paso a paso por qué la inferencia es intratable en una BM general y tratable en una RBM.

Para entender esto, primero definamos la **Función de Energía** y la **Probabilidad** básica que rige a ambos modelos.

La probabilidad de una configuración conjunta (v, h) está dada por la distribución de Boltzmann:

$$P(v, h) = \frac{1}{Z} e^{-E(v, h)}$$

Donde Z es la función de partición (la suma de todas las configuraciones posibles, necesaria para normalizar).

1. Máquina de Boltzmann General (BM) - El Caso Intratable

En una BM general, **todas las unidades están conectadas entre sí**, incluyendo las unidades ocultas entre ellas.

Paso 1: Definir la Energía

La función de energía para una BM incluye términos de interacción entre unidades ocultas (h_j y h_k):

$$E(v, h) = - \sum_i v_i b_i - \sum_j h_j c_j - \sum_{i,j} v_i h_j w_{ij} - \underbrace{\sum_{j,k} h_j h_k U_{jk}}_{\text{Problema: Interacción oculta-oculta}} - \sum_{i,k} v_i v_k V_{ik}$$

(Para simplificar la notación, ignoraremos los bias y las conexiones visible-visible, ya que el problema principal radica en las conexiones $h - h$).

Paso 2: Formular la Probabilidad Condicional

Queremos calcular $P(h_j = 1|v)$. Por definición de probabilidad condicional:

$$P(h_j = 1|v) = \frac{P(h_j = 1, v)}{P(v)}$$

Para obtener el numerador (y el denominador), necesitamos considerar **todas las posibles configuraciones de las otras unidades ocultas** (llamémoslas h_{-j} , que significa "todas las h excepto la j ").

$$P(h_j = 1|v) = \frac{\sum_{h_{-j}} e^{-E(v, h_j=1, h_{-j})}}{\sum_h e^{-E(v, h)}}$$

Paso 3: Análisis de la Suma

Observa el término de energía en el exponente. Debido al término $\sum_{j,k} h_j h_k U_{jk}$, el estado de h_j depende del estado de h_k .

Esto significa que la función exponencial **no se puede factorizar** limpiamente en productos independientes para cada unidad oculta.

Matemáticamente, **no** podemos hacer esto: $\sum_{h_1} \dots \sum_{h_N} \neq (\sum_{h_1}) \dots (\sum_{h_N})$.

Paso 4: Conclusión de Complejidad

Para calcular el numerador $\sum_{h_{-j}}$, debemos sumar sobre todas las combinaciones posibles de las otras unidades ocultas.

Si tenemos N unidades ocultas, hay 2^{N-1} configuraciones posibles.

- Si $N = 10$, son 1024 sumas.
- Si $N = 100$, son 2^{99} sumas (astronómico).

Resultado: La complejidad es **Exponencial** $O(2^N)$. Esto es computacionalmente **intratable** para redes de tamaño útil.

2. Máquina de Boltzmann Restringida (RBM) - El Caso Tratable

En una RBM, **no hay conexiones intra-capas**. Las unidades ocultas NO se conectan entre si ($U_{jk} = 0$).

Paso 1: Definir la Energía

La función de energía simplificada es:

$$E(v, h) = - \sum_i v_i b_i - \sum_j h_j c_j - \sum_{i,j} v_i h_j w_{ij}$$

Observa que no hay términos $h_j h_k$.

Paso 2: Factorización de la Probabilidad

Dado un vector visible v fijo, queremos calcular la probabilidad de todo el vector oculto h .

$$P(h|v) = \frac{1}{Z'} e^{-E(v,h)} = \frac{1}{Z'} e^{\sum_j h_j c_j + \sum_{i,j} v_i h_j w_{ij}}$$

(Nota: El término $\sum v_i b_i$ se cancela o se vuelve constante porque v está fijo).

Podemos reescribir el exponente agrupando por j :

$$P(h|v) = \frac{1}{Z'} e^{\sum_j h_j (c_j + \sum_i v_i w_{ij})}$$

Por propiedad de los exponenciales ($e^{a+b} = e^a e^b$), la suma en el exponente se convierte en un producto:

$$P(h|v) = \frac{1}{Z'} \prod_j e^{h_j (c_j + \sum_i v_i w_{ij})}$$

Paso 3: Independencia Condicional

Debido a que la probabilidad conjunta se factoriza en un producto ($\prod_j$), esto implica que las unidades ocultas son **condicionalmente independientes** dado v .

$$P(h|v) = P(h_1|v) \cdot P(h_2|v) \cdot \dots \cdot P(h_N|v)$$

Esto significa que para calcular $P(h_j = 1|v)$, **no necesitamos sumar sobre las otras unidades ocultas**, porque no dependen entre si.

Paso 4: Derivación Analítica (Cálculo directo)

Calculamos la probabilidad para una sola unidad h_j (binaria, toma valores 0 o 1). Sea $x_j = c_j + \sum_i v_i w_{ij}$ la entrada total a la unidad j .

$$P(h_j = 1|v) = \frac{P(h_j = 1, v)}{P(h_j = 0, v) + P(h_j = 1, v)}$$

(Usando los términos de energía no normalizados):

$$P(h_j = 1|v) = \frac{e^{1 \cdot x_j}}{e^{0 \cdot x_j} + e^{1 \cdot x_j}}$$

$$P(h_j = 1|v) = \frac{e^{x_j}}{1 + e^{x_j}}$$

Si dividimos numerador y denominador por e^{x_j} , obtenemos la función sigmoide:

$$P(h_j = 1|v) = \frac{1}{1 + e^{-x_j}} = \sigma \left(c_j + \sum_i v_i w_{ij} \right)$$

Paso 5: Conclusión de Complejidad

Para calcular el valor de una unidad oculta, solo necesitamos hacer una suma ponderada de las entradas visibles ($v_i w_{ij}$) y aplicar la función sigmoide.

Resultado: La complejidad es **Lineal** respecto al número de conexiones $O(M)$, donde M es el número de unidades visibles. No hay sumatoria exponencial. Esto es **tratable y eficiente**.

2. (RBM) Derivación de la Probabilidad Condicional

A partir de la función de energía de una RBM binaria:

$$E(v, h) = -a^T v - b^T h - v^T W h.$$

Derive formalmente la expresión para la probabilidad condicional de que una sola unidad oculta h_j esté activa, $P(h_j = 1|v)$, demostrando que es la función sigmoide.

1. Definiciones Iniciales

Partimos de la probabilidad conjunta definida por la distribución de Boltzmann:

$$P(v, h) = \frac{1}{Z} e^{-E(v, h)}$$

Y la función de energía dada:

$$E(v, h) = -a^T v - b^T h - v^T W h$$

Para facilitar la derivación respecto a una unidad específica h_j , expandiremos la notación vectorial a sumatorias

$$E(v, h) = - \sum_i a_i v_i - \sum_k b_k h_k - \sum_i \sum_k v_i W_{ik} h_k$$

2. Definición de la Probabilidad Condicional

Queremos calcular la probabilidad de que la unidad oculta j sea 1, dado el vector visible v .

Por definición de probabilidad condicional para variables binarias:

$$P(h_j = 1|v) = \frac{P(h_j = 1, v)}{P(h_j = 0, v) + P(h_j = 1, v)}$$

(Nota: Aquí omitimos la suma sobre las otras unidades ocultas h_{-j} porque, dado v , las unidades ocultas son independientes entre sí. Podemos tratar a las demás como constantes que se cancelarán).

Sustituyendo la definición de Boltzmann (Z se cancela en el numerador y denominador):

$$P(h_j = 1|v) = \frac{e^{-E(v, h_j=1)}}{e^{-E(v, h_j=0)} + e^{-E(v, h_j=1)}}$$

3. Separación de la Energía

Vamos a separar la función de energía en dos partes: los términos que dependen de h_j y los que no.

$$E(v, h) = \underbrace{\left(- \sum_i a_i v_i - \sum_{k \neq j} b_k h_k - \sum_i \sum_{k \neq j} v_i W_{ik} h_k \right)}_{\text{Términos que NO dependen de } h_j} - \underbrace{\left(b_j h_j + \sum_i v_i W_{ij} h_j \right)}_{\text{Términos que SÍ dependen de } h_j}$$

Llamemos C a la parte constante (que no depende de h_j) y definamos la "entrada total" a la neurona j como x_j :

$$x_j = b_j + \sum_i v_i W_{ij}$$

Entonces la energía se simplifica a:

$$E(v, h) = C - h_j \cdot x_j$$

4. Evaluación de los Estados de h_j

Ahora evaluamos esta energía simplificada para los dos posibles estados de h_j (0 y 1):

1. Si $h_j = 0$:

$$E(v, h_j = 0) = C - (0 \cdot x_j) = C$$

2. Si $h_j = 1$:

$$E(v, h_j = 1) = C - (1 \cdot x_j) = C - x_j$$

5. Sustitución en la Fórmula de Probabilidad

Sustituimos estas energías en la ecuación del paso 2:

$$P(h_j = 1|v) = \frac{e^{-(C-x_j)}}{e^{-C} + e^{-(C-x_j)}}$$

Aplicamos propiedades de los exponentes ($e^{-(A-B)} = e^{-A}e^B$):

$$P(h_j = 1|v) = \frac{e^{-C} \cdot e^{x_j}}{e^{-C} + e^{-C} \cdot e^{x_j}}$$

Factorizamos e^{-C} en el denominador:

$$P(h_j = 1|v) = \frac{e^{-C} \cdot e^{x_j}}{e^{-C}(1 + e^{x_j})}$$

Cancelamos el término e^{-C} (que representaba todo el resto de la red):

$$P(h_j = 1|v) = \frac{e^{x_j}}{1 + e^{x_j}}$$

6. Forma Final (Sigmoide)

Para llegar a la forma estándar de la sigmoide, dividimos numerador y denominador por e^{x_j} (o multiplicamos por e^{-x_j}/e^{-x_j}):

$$P(h_j = 1|v) = \frac{1}{e^{-x_j}(1 + e^{x_j})}$$

$$P(h_j = 1|v) = \frac{1}{e^{-x_j} + 1}$$

Reinsertando el valor de x_j :

$$P(h_j = 1|v) = \sigma \left(b_j + \sum_{i=1}^m v_i W_{ij} \right)$$

Q.E.D. (Queda demostrado).

4. (RBM) Derivación del Gradiente de la Log-Verosimilitud

El objetivo de entrenamiento de una RBM es maximizar la log-verosimilitud de los datos visibles,

$$\mathcal{L} = \log P(v).$$

Demuestre que la derivada de esta función con respecto a un peso W_{ij} es:

$$\frac{\partial \log P(v)}{\partial W_{ij}} = \langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}}$$

Sugerencia: Utilice el "truco log-derivativo" en $\frac{\partial \log Z}{\partial W_{ij}}$ donde Z es la función de partición.

1. Definición del Objetivo

Queremos derivar la función de Log-Verosimilitud ($\mathcal{L}$) con respecto a un peso específico W_{ij} .

$$\mathcal{L} = \log P(v) = \log \left(\frac{1}{Z} \sum_h e^{-E(v,h)} \right)$$

Usando la propiedad del logaritmo de una división ($\log \frac{A}{B} = \log A - \log B$), separamos la ecuación en dos términos:

$$\mathcal{L} = \underbrace{\log \left(\sum_h e^{-E(v,h)} \right)}_{\text{Término 1}} - \underbrace{\log Z}_{\text{Término 2}}$$

Donde $Z = \sum_{v,h} e^{-E(v,h)}$.

2. Derivada de la Energía

Antes de derivar los logaritmos, necesitamos saber cómo cambia la energía con respecto al peso W_{ij} .

La función de energía es:

$$E(v, h) = - \sum a_v - \sum b_h - \sum_{k,l} v_k h_l W_{kl}$$

La derivada parcial con respecto a W_{ij} solo afecta al término donde aparecen i y j :

$$\frac{\partial E(v, h)}{\partial W_{ij}} = -(v_i h_j)$$

3. Derivación del Término 1 (Fase Positiva)

Este término representa la probabilidad de los datos observados. Usamos la regla de la cadena para la derivada del logaritmo ($\frac{\partial \log u}{\partial x} = \frac{1}{u} \frac{\partial u}{\partial x}$):

$$\frac{\partial}{\partial W_{ij}} \log \left(\sum_h e^{-E(v,h)} \right) = \frac{1}{\sum_h e^{-E(v,h)}} \cdot \frac{\partial}{\partial W_{ij}} \left(\sum_h e^{-E(v,h)} \right)$$

Derivamos la sumatoria:

$$= \frac{1}{\sum_h e^{-E(v,h)}} \sum_h \left(e^{-E(v,h)} \cdot \frac{-\partial E(v, h)}{\partial W_{ij}} \right)$$

Sustituimos la derivada de la energía ($-\partial E(v, h) = v_i h_j$):

$$= \frac{\sum_h e^{-E(v,h)} (v_i h_j)}{\sum_h e^{-E(v,h)}}$$

Observa que $\frac{e^{-E(v,h)}}{\sum_h e^{-E(v,h)}}$ es exactamente la probabilidad condicional $P(h|v)$. Por lo tanto, este término es el valor esperado de $v_i h_j$ dado el dato de entrada v :

$$\text{Término 1} = \sum_h P(h|v) \cdot v_i h_j = \langle v_i h_j \rangle_{\text{data}}$$

4. Derivación del Término 2 (Fase Negativa)

Este término involucra la función de partición Z . Usamos la sugerencia del "truco log-derivativo":

$$\frac{\partial}{\partial W_{ij}} \log Z = \frac{1}{Z} \frac{\partial Z}{\partial W_{ij}}$$

Recordemos que Z suma sobre **todas** las configuraciones posibles de visibles y ocultas (v', h') :

$$Z = \sum_{v', h'} e^{-E(v', h')}$$

Calculamos la derivada de Z :

$$\frac{\partial Z}{\partial W_{ij}} = \sum_{v', h'} e^{-E(v', h')} (v'_i h'_j)$$

Sustituimos de nuevo en la ecuación logarítmica:

$$\frac{\partial \log Z}{\partial W_{ij}} = \sum_{v', h'} \frac{e^{-E(v', h')}}{Z} (v'_i h'_j)$$

Aquí, $\frac{e^{-E(v', h')}}{Z}$ es la probabilidad conjunta $P(v', h')$ definida por el modelo. Por lo tanto, esto es el valor esperado sobre todas las configuraciones posibles del modelo:

$$\text{Término 2} = \sum_{v', h'} P(v', h') \cdot v'_i h'_j = \langle v_i h_j \rangle_{\text{model}}$$

5. Resultado Final

Unimos ambos términos (recordando el signo menos entre ellos del Paso 1):

$$\frac{\partial \log P(v)}{\partial W_{ij}} = \text{Término 1} - \text{Término 2}$$

$$\frac{\partial \log P(v)}{\partial W_{ij}} = \langle v_i h_j \rangle_{\text{data}} - \langle v_i h_j \rangle_{\text{model}}$$

Interpretación:

El gradiente nos dice que debemos ajustar los pesos para **aumentar** la correlación entre v y h cuando el modelo ve datos reales (data), y **disminuir** la correlación cuando el modelo "sueña" o genera datos por sí mismo (model).

5. (CD-k) Justificación Teórica de la Divergencia Contrastiva

Explique conceptual y matemáticamente por qué el término $\langle v_i h_j \rangle_{\text{model}}$ derivado en el ejercicio 4 es intratable. Luego, describa cómo el algoritmo de Divergencia Contrastiva de k pasos (CD- k), específicamente CD-1, utiliza el Muestreo de Gibbs para *aproximar* este término intratable.

1. ¿Por qué es Intratable el Término $\langle v_i h_j \rangle_{\text{model}}$?

El término $\langle v_i h_j \rangle_{\text{model}}$ representa la esperanza (el promedio) de la correlación entre las unidades visibles y ocultas bajo la distribución que el modelo "imagina" o genera por sí mismo.

Explicación Matemática

Matemáticamente, este valor esperado se define como la suma ponderada de todas las configuraciones posibles:

$$\langle v_i h_j \rangle_{\text{model}} = \sum_{v, h} P(v, h) \cdot v_i h_j$$

Sustituyendo $P(v, h) = \frac{e^{-E(v, h)}}{Z}$, obtenemos:

$$\langle v_i h_j \rangle_{\text{model}} = \frac{1}{Z} \sum_{v \in \{0,1\}^m} \sum_{h \in \{0,1\}^n} e^{-E(v, h)} \cdot v_i h_j$$

Aquí radica el problema:

1. **La Sumatoria Exponencial:** Para calcular esto exactamente, debemos sumar sobre **todas** las combinaciones posibles de vectores visibles (v) y ocultos (h).
2. **La Función de Partición (Z):** Incluso para calcular solo Z (el denominador), necesitamos la misma suma monstruosa: $Z = \sum_{v, h} e^{-E(v, h)}$.

Explicación Conceptual

Si tienes una RBM pequeña con $m = 100$ visibles y $n = 100$ ocultas, el número total de estados posibles es 2^{200} .

- $2^{200} \approx 1.6 \times 10^{60}$ configuraciones.

Para obtener el valor exacto, tendrías que iterar sobre más estados que átomos hay en el sistema solar. Ninguna computadora puede realizar esa suma en un tiempo razonable. Por eso se dice que es **computacionalmente intratable**.

2. La Solución: Divergencia Contrastiva (CD-k) y Muestreo de Gibbs

Como no podemos calcular la suma exacta, recurrimos a la estadística: **estimamos el promedio usando muestras**.

El algoritmo estándar para generar muestras de una distribución conjunta compleja es la **Cadena de Markov Monte Carlo (MCMC)**, específicamente el **Muestreo de Gibbs**.

Cómo funciona el Muestreo de Gibbs en una RBM

Gracias a la estructura bipartita de la RBM (no hay conexiones intra-capas), podemos muestrear fácilmente en "zig-zag":

1. Dado un v , obtenemos h muestreando de $P(h|v)$.
2. Dado ese h , obtenemos un nuevo v muestreando de $P(v|h)$.

Si repetimos esto infinitas veces ($k \rightarrow \infty$), la cadena converge a la distribución real del modelo (el equilibrio térmico) y obtendríamos una muestra perfecta para calcular $\langle v_i h_j \rangle_{\text{model}}$. **Problema:** Tardaría demasiado en converger.

La Aproximación CD-1

Geoffrey Hinton demostró que no necesitamos esperar a que la cadena converja al equilibrio perfecto para obtener una buena dirección de aprendizaje. Basta con dar **unos pocos pasos** (k) desde los datos reales.

En **CD-1** ($k = 1$), hacemos lo siguiente para aproximar el término intratable:

1. **Inicialización** (v_0): Empezamos la cadena en un dato real de entrenamiento. (Esto es clave: empezamos en un lugar de alta probabilidad según los datos, no en un lugar aleatorio).
2. **Paso Adelante** (h_0): Calculamos las probabilidades ocultas y muestreamos $h_0 \sim P(h|v_0)$.
3. **Paso Atrás / Reconstrucción** (v_1): Generamos una "fantasía" o reconstrucción visible muestreando $v_1 \sim P(v|h_0)$.
4. **Paso Adelante Final** (h_1): Calculamos las nuevas activaciones ocultas basadas en la fantasía $h_1 \sim P(h|v_1)$.

La Aproximación Final

En lugar de la suma infinita intratable, usamos simplemente el producto de esta reconstrucción de un solo paso:

$$\langle v_i h_j \rangle_{\text{model}} \approx v_{1,i} \cdot h_{1,j}$$

(Nota: En la práctica, para reducir el ruido, a menudo se usan las probabilidades reales en lugar de los valores binarios muestreados en el último paso).

Resumen

- **Problema:** Calcular el término del modelo requiere sumar 2^{N+M} términos, lo cual es imposible.
- **Solución (Gibbs):** Navegar por el espacio de estados alternando entre v y h .
- **Truco (CD-1):** En lugar de ejecutar Gibbs hasta el infinito, damos **un solo paso** ($v_0 \rightarrow h_0 \rightarrow v_1 \rightarrow h_1$). Asumimos que la dirección hacia la que se mueve el modelo después de un paso es una buena aproximación de hacia dónde quiere ir el modelo para minimizar su energía.

7. (Autoencoder) Demostración de Equivalencia con PCA

Demuestre que un Autoencoder *lineal* (es decir, $f(z) = z$ para todas las activaciones) con una sola capa oculta y entrenado para minimizar el Error Cuadrático Medio (MSE)

$$L = ||x - x'||^2,$$

aprende a proyectar los datos en el mismo subespacio vectorial que el Análisis de Componentes Principales (PCA).

Aquí tienes la demostración resumida paso a paso:

1. Definición del Modelo

En un Autoencoder Lineal, tanto el codificador como el decodificador son transformaciones lineales simples (matrices), sin funciones de activación no lineales.

- Codificador:** $z = xW_1$ (proyecta de dimensión d a k).
- Decodificador:** $x' = zW_2$ (proyecta de k a d).
- Reconstrucción total:** $x' = x(W_1W_2)$.

Definamos la matriz combinada $W = W_1W_2$. Dado que la información pasa por un cuello de botella de tamaño k (donde $k < d$), la matriz W tiene forzosamente **rango** k .

2. El Objetivo de Optimización

Entrenar el Autoencoder minimizando el Error Cuadrático Medio (MSE) equivale a resolver el siguiente problema de optimización sobre toda la matriz de datos X :

$$\min_W ||X - XW||_F^2 \quad \text{sujeto a } \text{rango}(W) \leq k$$

Donde $|| \cdot ||_F$ es la norma de Frobenius. Buscamos la mejor aproximación de rango k de la matriz identidad aplicada a los datos.

3. El Teorema de Eckart-Young

Según el teorema de **Eckart-Young-Mirsky** (un resultado fundamental en álgebra lineal), la matriz de rango k que minimiza el error de reconstrucción (norma de Frobenius) de una matriz de datos X se obtiene mediante la **Descomposición en Valores Singulares (SVD)** truncada.

Específicamente, la solución óptima es proyectar los datos sobre el subespacio generado por los k **vectores singulares (o vectores propios) principales** de la matriz de covarianza de los datos ($X^T X$).

4. Conclusión (La Equivalencia)

- PCA** se define precisamente como la proyección ortogonal de los datos sobre los vectores propios principales de la covarianza para minimizar el error cuadrático.
- Como el **Autoencoder Lineal** está minimizando exactamente el mismo error cuadrático con una restricción de rango k , la solución óptima de los pesos del Autoencoder debe generar el **mismo subespacio** que PCA.

Resumen: Al minimizar el MSE, el Autoencoder Lineal "descubre" que la mejor manera de comprimir y recuperar datos linealmente es proyectarlos sobre sus Componentes Principales.

Nota técnica: Los pesos W_1 aprendidos pueden no ser exactamente los vectores propios de PCA, pero **expanden el mismo subespacio** (pueden ser una rotación de los componentes principales).

8. (Autoencoder) Derivación de Backpropagation (Capa Encoder)

Considere un autoencoder simple ($x \rightarrow h \rightarrow x'$) con funciones de activación sigmoide ($\sigma(z)$) en ambas capas. La pérdida es el Error Cuadrático Medio

$$L = \frac{1}{2} \sum_k (x_k - x'_k)^2.$$

- $z_h = xW_e + b_e$ (Entrada a la capa oculta)
- $h = \sigma(z_h)$ (Salida de la capa oculta)
- $z_o = hW_d + b_d$ (Entrada a la capa de salida)
- $x' = \sigma(z_o)$ (Salida de reconstrucción)

Derive la expresión completa para el gradiente de la pérdida con respecto a un peso en la capa encoder, $\frac{\partial L}{\partial (W_e)_{ij}}$.

1. Paso 1: El error en la Capa de Salida (δ_o)

Primero calculamos cómo cambia el error respecto a la entrada neta de la capa de salida (z_o).

- Derivada del Error: $\frac{\partial L}{\partial x'_k} = -(x_k - x'_k)$
- Derivada de la Sigmoide: $\frac{\partial x'_k}{\partial z_{o,k}} = x'_k(1 - x'_k)$

Multiplicando ambos, obtenemos el término de error de salida ($\delta_{o,k}$):

$$\delta_{o,k} = \frac{\partial L}{\partial z_{o,k}} = -(x_k - x'_k) \cdot x'_k(1 - x'_k)$$

2. Paso 2: Retropropagación a la Capa Oculta (δ_h)

Ahora traemos ese error hacia atrás, a través de los pesos del decodificador (W_d), hasta la entrada neta de la capa oculta (z_h).

- Suma ponderada de errores: $\sum_k \delta_{o,k} (W_d)_{jk}$
- Derivada de la Sigmoide oculta: $h_j(1 - h_j)$

Multiplicando, obtenemos el término de error oculto ($\delta_{h,j}$):

$$\delta_{h,j} = \left(\sum_k \delta_{o,k} (W_d)_{jk} \right) \cdot h_j(1 - h_j)$$

3. Paso 3: Gradiente respecto al Peso del Encoder

Finalmente, conectamos el error oculto con el peso específico (W_e)_{ij}.

- La entrada a la neurona oculta es $z_{h,j} = \dots + x_i(W_e)_{ij} + \dots$
- La derivada parcial es simplemente la entrada asociada: $\frac{\partial z_{h,j}}{\partial (W_e)_{ij}} = x_i$

Resultado Final

Combinando todo (Gradiente = Error Local $\times$ Entrada):

$$\frac{\partial L}{\partial (W_e)_{ij}} = \delta_{h,j} \cdot x_i$$

Expandido completamente:

$$\frac{\partial L}{\partial (W_e)_{ij}} = \left[\left(\sum_k \underbrace{-(x_k - x'_k)x'_k(1 - x'_k)}_{\delta_{o,k}} (W_d)_{jk} \right) \cdot \underbrace{h_j(1 - h_j)}_{\sigma'(z_h)} \right] \cdot x_i$$

9. (Integración RBM-AE) Arquitectura de Pre-entrenamiento y Ajuste Fino

Basado en la clase, explique el proceso de 3 etapas para entrenar un Autoencoder Profundo (Stacked Autoencoder) usando RBMs:

1. **Pre-entrenamiento:** ¿Cómo se entrenan las RBMs (W_1, W_2) de forma codiciosa (greedy layer-wise)?
2. **Desdoblamiento (Unfolding):** ¿Cómo se inicializan los pesos del AE ($W_{e1}, W_{e2}, W_{d1}, W_{d2}$) a partir de W_1 y W_2 ?

3. **Ajuste Fino (Fine-Tuning):** ¿Cuál es la función de pérdida y el algoritmo utilizado en esta etapa final?

1. Pre-entrenamiento (Greedy Layer-wise)

Se entrena la red "capa por capa" de abajo hacia arriba, tratando cada par de capas como una RBM independiente.

- **Paso 1:** Se entrena la **RBM 1** con los datos de entrada originales V para aprender W_1 .
- **Paso 2:** Se congelan los pesos de la RBM 1 y se pasan los datos a través de ella para obtener las activaciones ocultas.
- **Paso 3:** Se usan esas activaciones como "entrada visible" para entrenar la **RBM 2** y aprender W_2 .
(Es "codicioso" porque cada capa intenta optimizar su propia representación sin preocuparse por las demás).

2. Desdoblamiento (Unfolding)

Se convierte la pila de RBMs probabilísticas en una red determinista (Encoder-Decoder) simétrica inicializando los pesos así:

- **Encoder (Codificador):** Se usan los pesos originales aprendidos.
 - $W_{e1} = W_1$
 - $W_{e2} = W_2$
- **Decoder (Decodificador):** Se usan las **transpuestas** de esos mismos pesos.
 - $W_{d2} = W_2^T$
 - $W_{d1} = W_1^T$

3. Ajuste Fino (Fine-Tuning)

Ahora que la red está inicializada en un "buen lugar" del espacio de parámetros, se entrena toda la red conjuntamente como un solo modelo determinista.

- **Función de Pérdida:** Generalmente **Error Cuadrático Medio (MSE)** o Entropía Cruzada, comparando la entrada original con la reconstrucción final (*Input* vs *Output*).
- **Algoritmo: Backpropagation** (con Descenso de Gradiente). Esto ajusta ligeramente todos los pesos (W_e y W_d) para minimizar el error global de reconstrucción.

10. (Síntesis) Comparación de Objetivos (RBM vs. AE)

Compare y contraste fundamentalmente los objetivos de entrenamiento de una RBM y un Autoencoder. ¿Por qué una RBM, que es esencialmente una red de 2 capas, requiere un algoritmo de aproximación complejo (Divergencia Contrastiva), mientras que un Autoencoder (potencialmente mucho más profundo) puede entrenarse directamente con Backpropagation?

1. Diferencia Fundamental: La Naturaleza del Modelo

Para entender los algoritmos, primero debemos entender qué es cada modelo:

- **Autoencoder (AE) - Modelo Determinista:**
 - Es una función matemática directa: $Output = f(Input)$.
 - Dada una entrada, **siempre** produce la misma salida.
 - No intenta modelar la probabilidad de los datos, solo intenta comprimirlos y descomprimirlos.
- **RBM - Modelo Probabilístico (Generativo):**
 - Define una **distribución de probabilidad** conjunta $P(v, h)$.
 - Su objetivo es aprender la "forma" de los datos en el espacio.
 - Dada una entrada, la respuesta es estocástica (depende de probabilidades).

2. Comparación de Objetivos de Entrenamiento

Autoencoder: Minimizar el Error de Reconstrucción

El objetivo del AE es geométrico. Quiere que la salida sea lo más parecida posible a la entrada.

$$Loss = ||v - \text{Decoder}(\text{Encoder}(v))||^2$$

- **Por qué usa Backpropagation:** Como el AE es una cadena de funciones deterministas (multiplicaciones de matrices y activaciones como ReLU o Sigmoid), podemos usar la **Regla de la Cadena** de cálculo. Podemos calcular exactamente cuánto contribuye cada peso al error final y ajustarlo. Es cálculo diferencial puro.

RBM: Maximizar la Verosimilitud (Log-Likelihood)

El objetivo de la RBM es estadístico. Quiere ajustar sus parámetros para que la distribución de probabilidad del modelo asigne la **máxima probabilidad posible a los datos de entrenamiento**.

$$\text{Maximizar } P(v) = \frac{1}{Z} \sum_h e^{-E(v,h)}$$

- **El Problema del Gradiente:** Cuando intentamos calcular el gradiente (la derivada) para maximizar esta probabilidad, surge una fórmula con dos términos:

$$\frac{\partial \log P(v)}{\partial w_{ij}} = \underbrace{\langle v_i h_j \rangle_{\text{datos}}}_{\text{Fase Positiva}} - \underbrace{\langle v_i h_j \rangle_{\text{modelo}}}_{\text{Fase Negativa}}$$

1. **Fase Positiva:** Fácil. Es lo que vemos en los datos.
2. **Fase Negativa:** Aquí está el problema. Este término requiere calcular el promedio sobre **todas las posibles configuraciones** que el modelo *podría* generar (el término de normalización Z o función de partición).

3. ¿Por qué la RBM necesita Divergencia Contrastiva (CD)?

La RBM requiere CD porque **no podemos calcular la derivada exacta**.

- 1. **El problema de la partición (Z):** Para saber la probabilidad real de un dato en una RBM, necesitas dividir por la suma de la energía de *todas* las combinaciones posibles de variables. Si tienes 100 neuronas, hay 2^{100} combinaciones. Sumar eso es imposible para el universo.
- 2. **La solución de CD:** Como no podemos calcular la "Fase Negativa" (el comportamiento global del modelo) exactamente, la **aproximamos**.
 - En lugar de sumar todo, tomamos **muestras** representativas usando la Cadena de Markov (Gibbs Sampling).
 - "Contrastive Divergence" es básicamente un truco para estimar esa integral imposible mediante un muestreo rápido (k pasos).

Resumen: ¿Por qué uno es "difícil" y el otro "fácil"?

Característica	Autoencoder (AE)	RBM
Meta	Mapear entrada → salida.	Modelar la densidad $P(x)$.
Matemática	Composición de funciones ($f(g(x))$).	Distribución de Boltzmann ($\frac{1}{Z}e^{-E}$).
Cálculo del Error	Local y Exacto: Solo comparas <i>Output</i> vs <i>Input</i> .	Global e Intratable: Comparas el dato vs. <i>todas las configuraciones posibles del universo del modelo (Z)</i> .
Solución	Backpropagation: La derivada fluye hacia atrás perfectamente.	Divergencia Contrastiva: Aproximación estocástica (muestreo) porque la derivada exacta es imposible de calcular.

En conclusión: Un Autoencoder puede ser muy profundo, pero sigue siendo un camino de "ida y vuelta" determinista. Una RBM, aunque tenga solo 2 capas, intenta resolver un problema de termodinámica estadística (normalizar una distribución de energía), lo cual es computacionalmente mucho más complejo que simplemente calcular derivadas parciales.